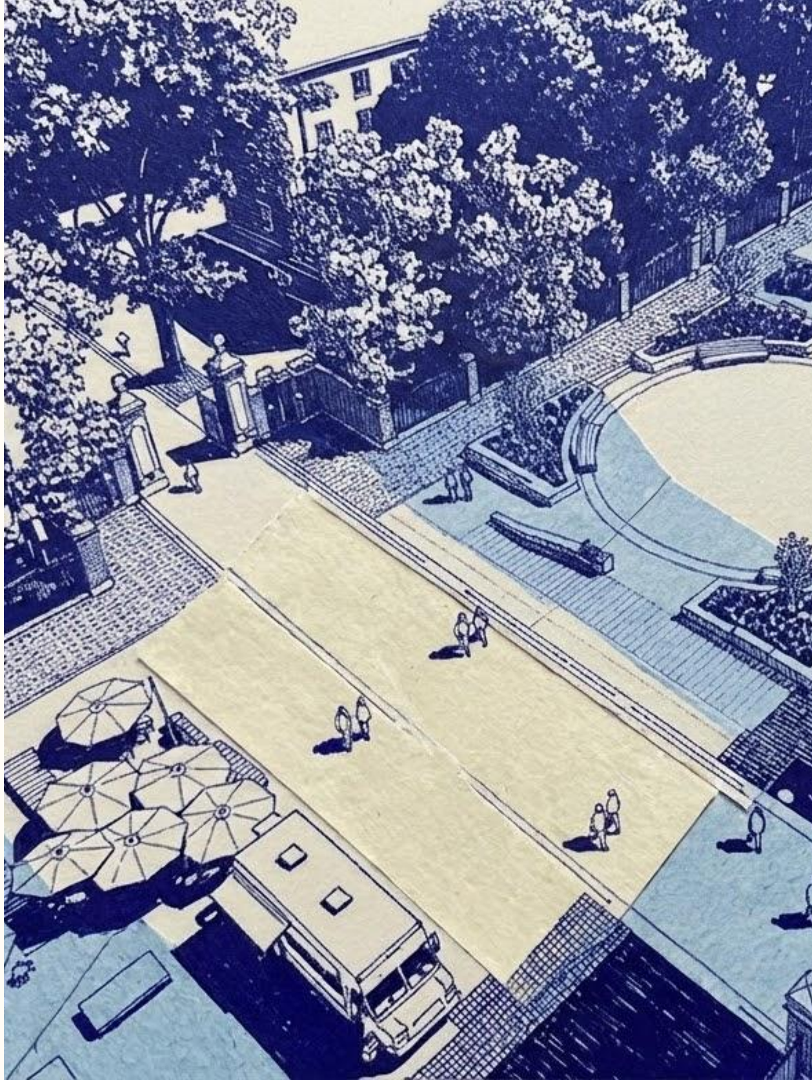
Science Center Plaza

PUBLICS IN *Motion*

Data-Informed Public Space Design through Motion Analysis



SCI 6506 | Mariana Ferreira & Raine Sha
Professor Charu Srivastava, Sang Wang, Sophia Chen

INDEX

1
Research:
Survey + Initial
Analysis

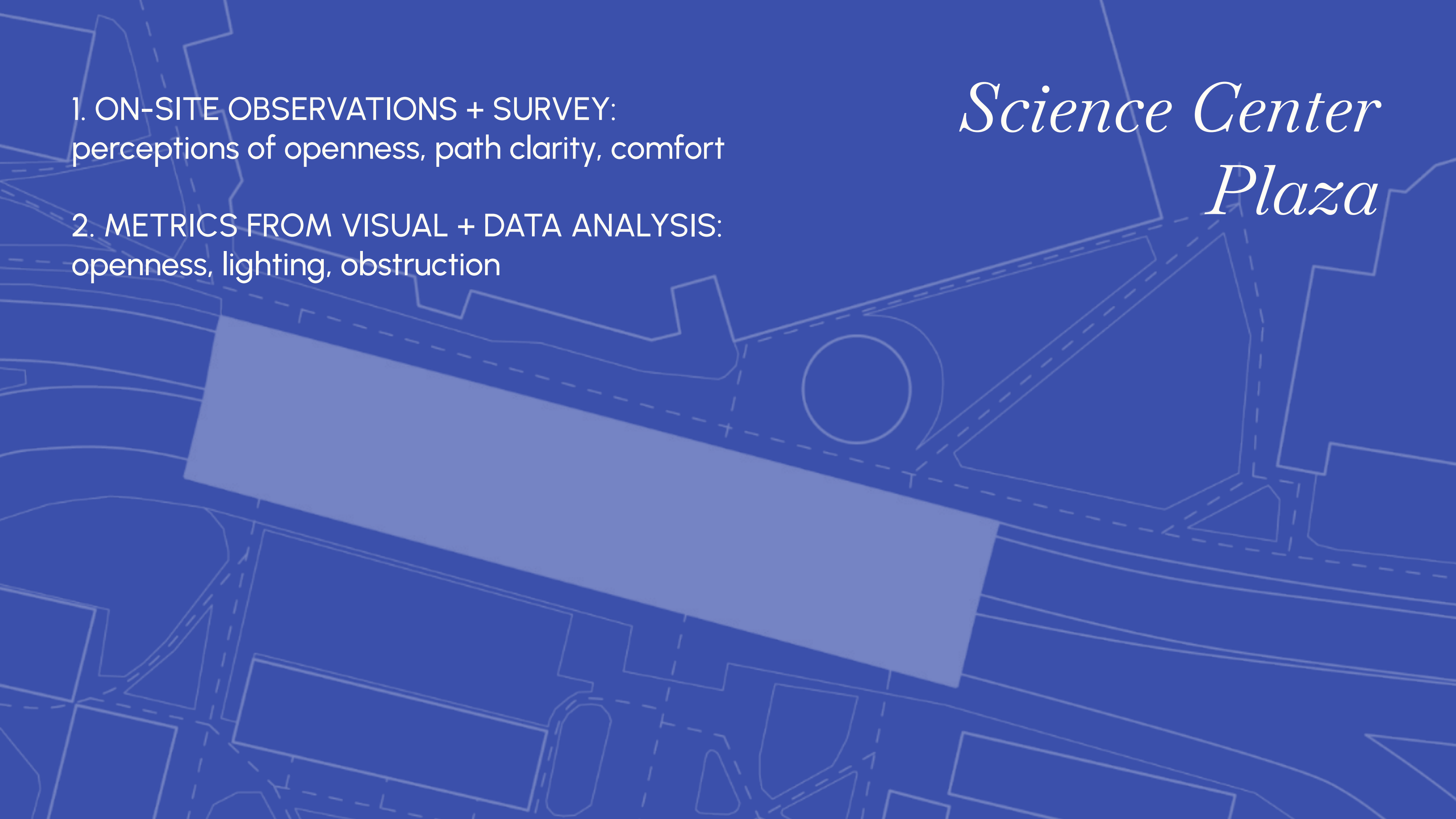
2
Video
Data
Collection

3
Computational
Analysis
and Scene
Typology

4
Motion Data
to Spatial
Analysis

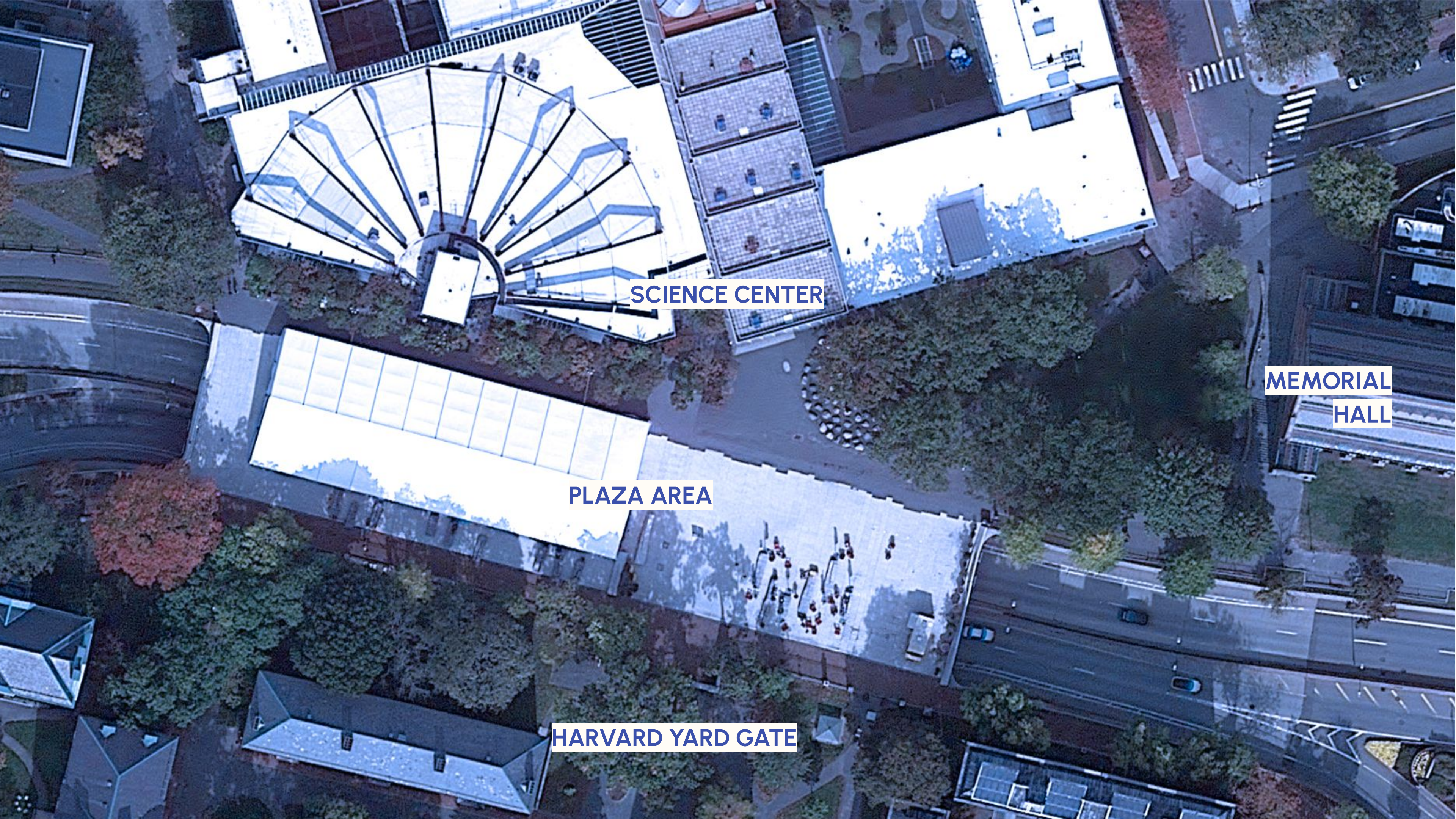
5
Data
Becomes
Design

Science Center Plaza



1. ON-SITE OBSERVATIONS + SURVEY:
perceptions of openness, path clarity, comfort

2. METRICS FROM VISUAL + DATA ANALYSIS:
openness, lighting, obstruction


SCIENCE CENTER
MEMORIAL HALL
PLAZA AREA
HARVARD YARD GATE

PERCEIVED ASPECTS

KINETIC PATTERNS

motion analysis

PERCEIVED ASPECTS

KINETIC PATTERNS

DESIGN INTERVENTIONS

motion analysis + behavioral patterns

How can computer-vision models *quantify and classify human movement speeds* in the Science Center Plaza?

How does *data relate to perceptions of openness, crowding, and comfort* captured in our previous surveys?

INDEX

1
Research:
Survey + Initial
Analysis

2
Video
Data
Collection

3
Computational
Analysis
and Scene
Typology

4
Motion Data
to Spatial
Analysis

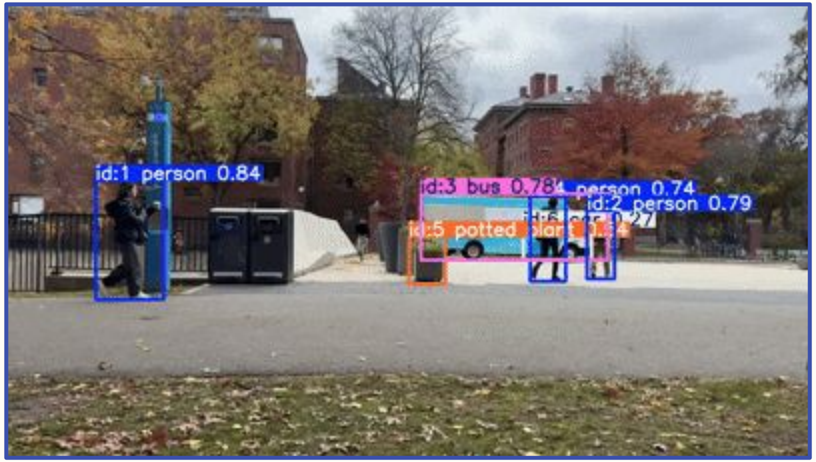
5
Data
Becomes
Design

5 key zones and multiple time windows

INDEX


id:1 person 0.87
id:296 person 0.38
id:237
id:29 person 0.68
truck
person 0.61

COMPUTATIONAL ANALYSIS



video tracking > top down projection > conversion to speed > classification

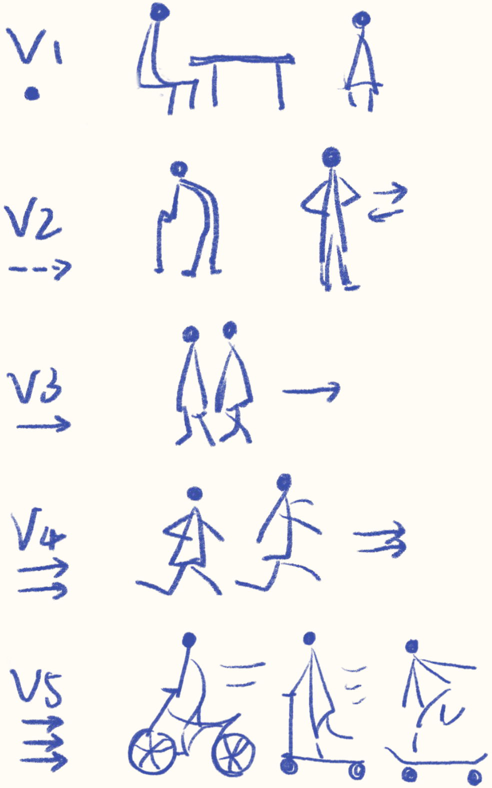
STAYING
< 0.2 m/s

SLOW WALKING
0.2–1.2 m/s

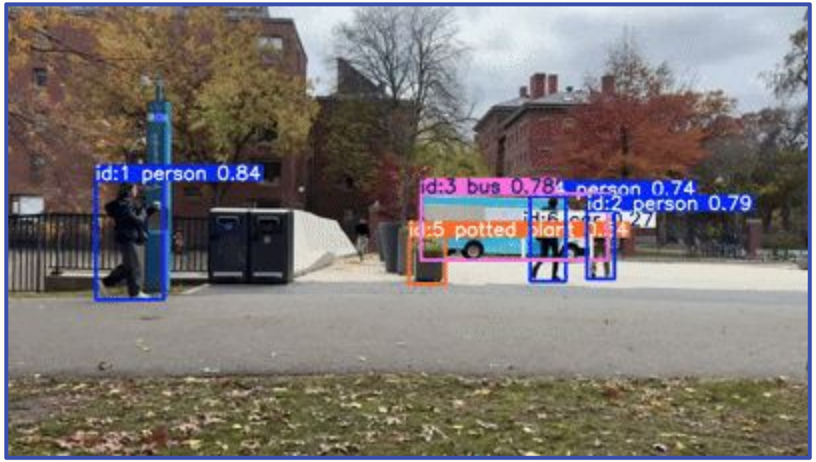
NORMAL WALKING
1.2–2.0 m/s

FAST WALKING / RUNNING
> 2.0 m/s

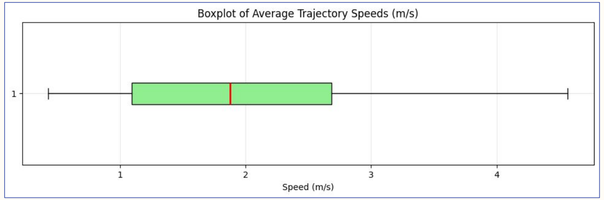
CYCLING / RIDING
> 3.0 m/s

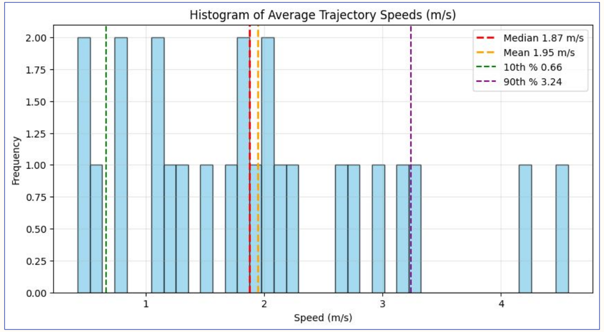


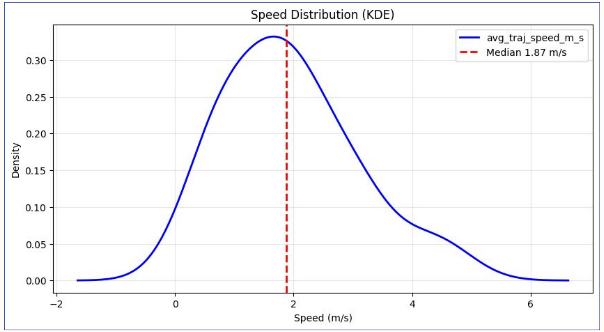
COMPUTATIONAL ANALYSIS



>>>



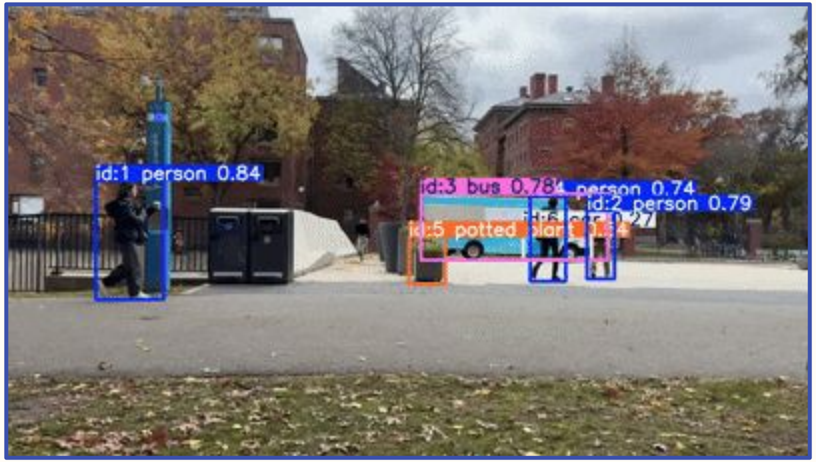




memorial hall corridor

Mean Speed: 1.95 m/s | Median: 1.87 m/s
Dominant Category: V3 (Normal walking)
with pockets of V4 (Fast walking)

COMPUTATIONAL ANALYSIS



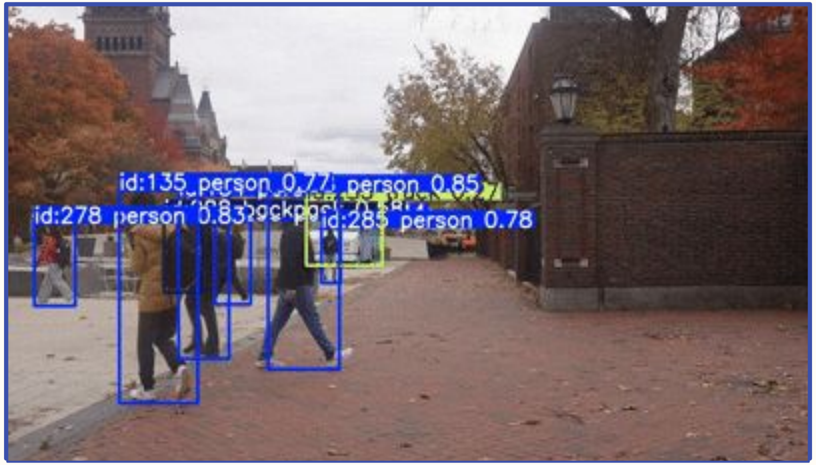
memorial hall corridor

This scene suggests a strong, consistent desire line. Its fast and unified flow also indicates potential conflict with any slow or lingering activity near the entrance.

>>>

BEHAVIORAL SIGNATURE

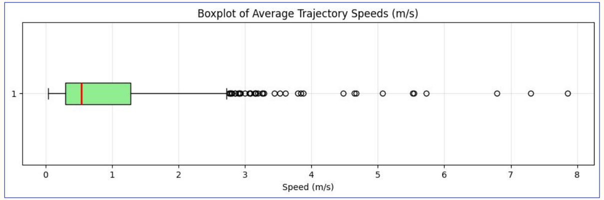
A high-throughput movement corridor, functioning primarily as a passing and commuting zone.

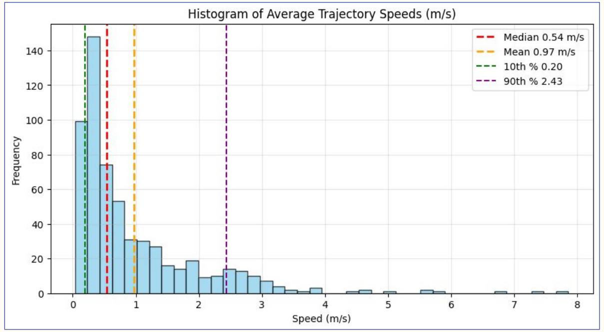


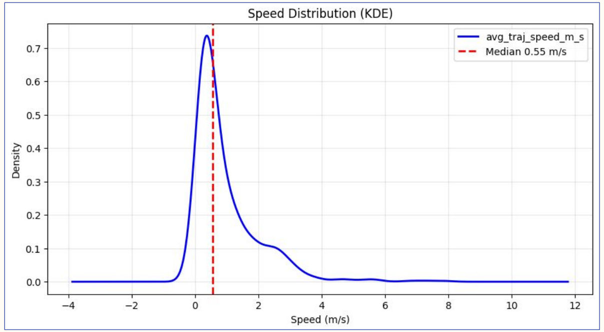
harvard yard entrance (lateral)

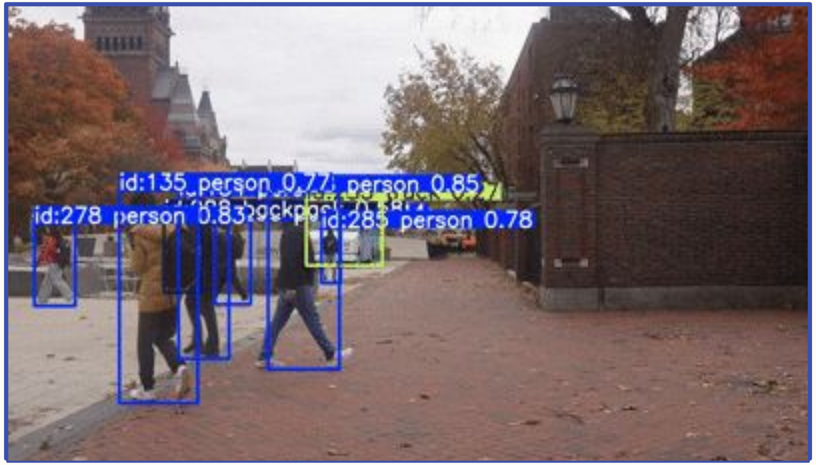
Mean speed: 0.95 m/s | Median: 0.54 m/s
Dominant Category: V1 - V2
(Slow movement)

>>>









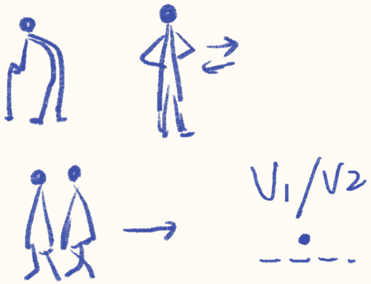
harvard yard entrance (lateral)

The gate acts as a funnel, compressing flows
and producing micro-congestion.
If high-speed flows from nearby paths intersect
this zone, conflicts become more likely.

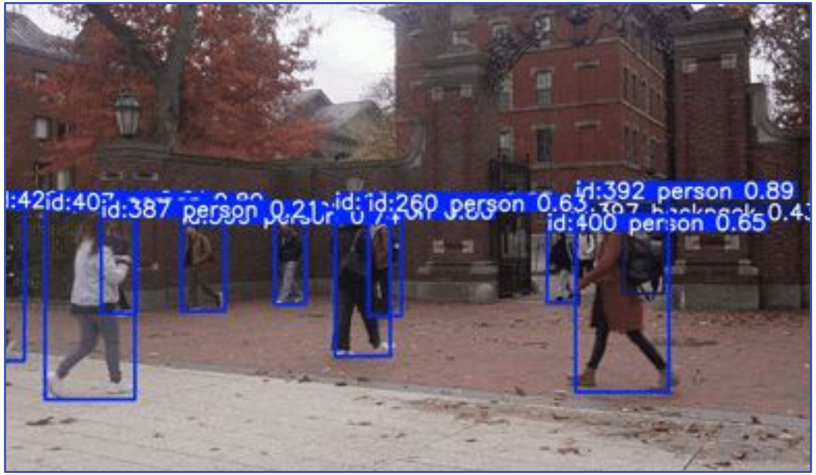
>>>

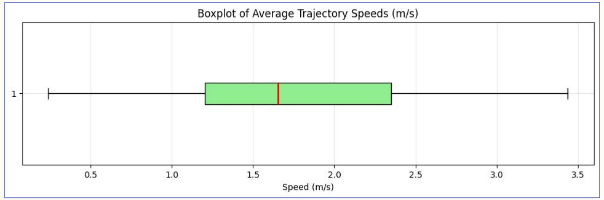
BEHAVIORAL SIGNATURE

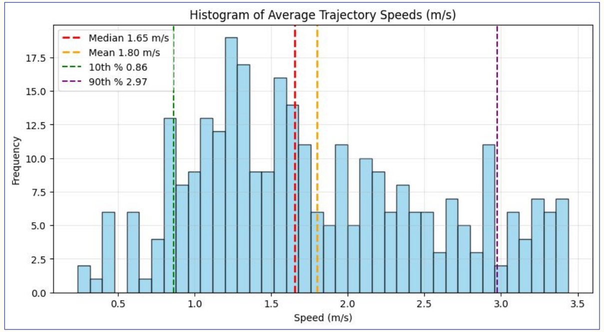
Transit bottleneck, where people naturally decelerate to pass through Harvard Yard Gate. The slow speeds reflect forced geometric constraint, not social staying.

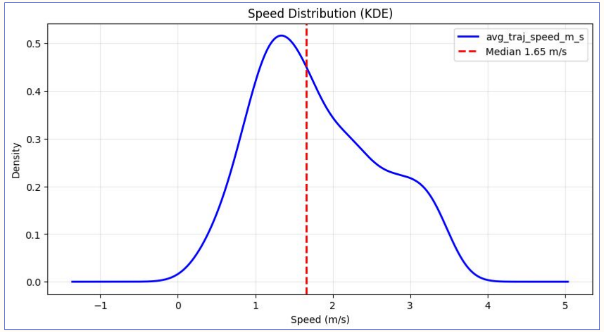


COMPUTATIONAL ANALYSIS







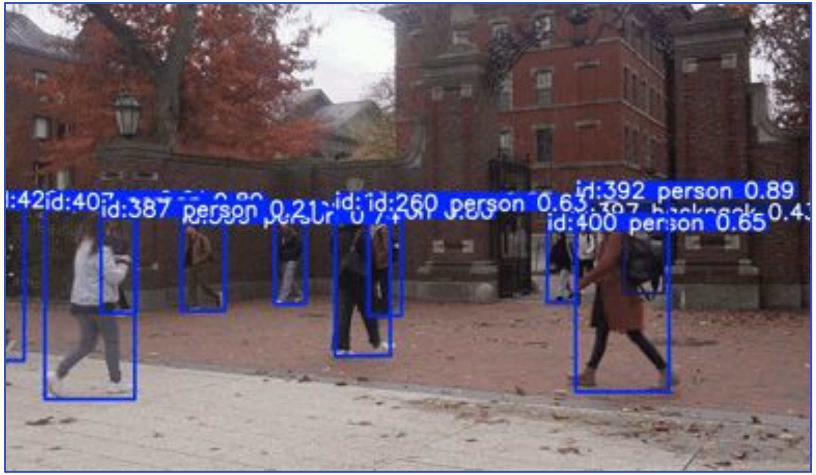


>>>

harvard yard entrance (frontal)

Mean speed: 1.80 m/s | Median: 1.65 m/s
Dominant Category: V3 (Normal walk)
Balanced; no extreme outliers.

COMPUTATIONAL ANALYSIS



harvard yard entrance (frontal)

This is a “middle-speed landscape” where flexible programming could work.

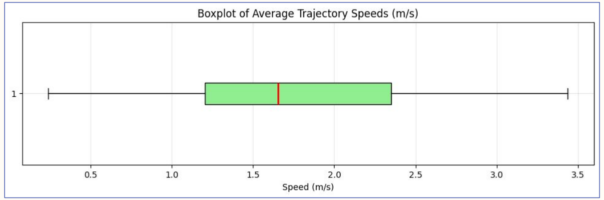
>>>

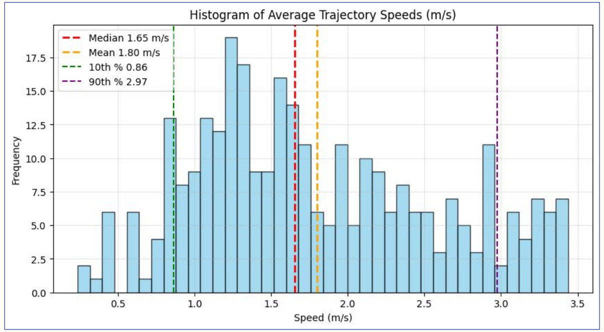
BEHAVIORAL SIGNATURE

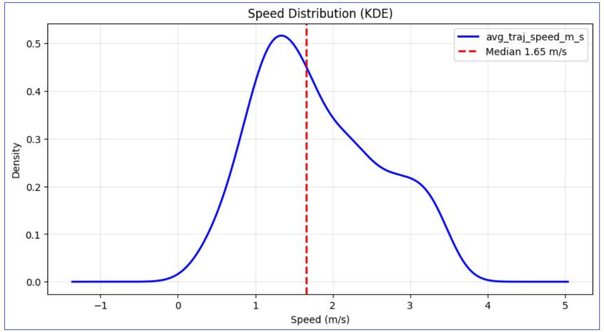
A stable transitional zone, neither purely stay-oriented nor a high-speed corridor.

COMPUTATIONAL ANALYSIS

>>>







science center main plaza

Mean speed: 2.10 m/s | Median: 2.00 m/s

Dominant Category: V3-V4

(Normal walk to fast walk, clustered at 1.5 - 3m/s, consistent forward motion)

COMPUTATIONAL ANALYSIS

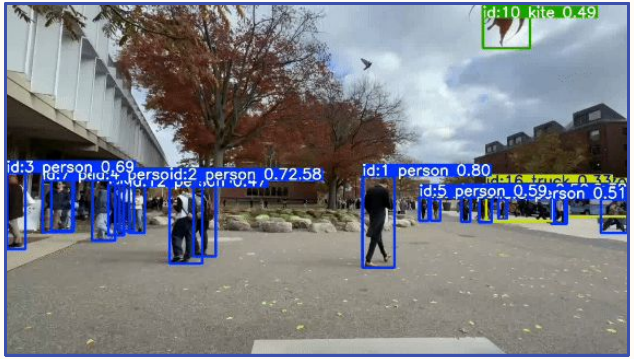
science center main plaza

Role as the primary desire line.
Design should support fast movement here, clarify direction, and reduce lateral interference.

>>>

BEHAVIORAL SIGNATURE

The commuter-oriented spine of the plaza.



science center entrance

daytime plaza

Median: 1.35 m/s | Mean: 1.71 m/s

Dominant category: V2–V3
(Slow walk to normal walk)
Long right tail; presence of extremely fast outliers (cyclists or fast walkers).

nighttime corridor

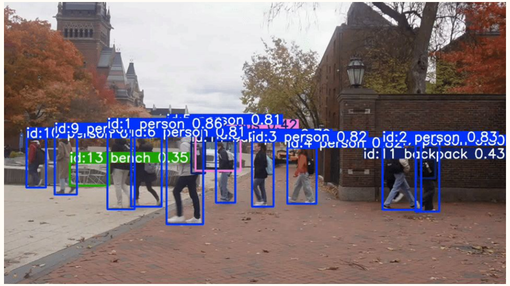
Median: 1.69 m/s | Mean: 1.97 m/s

Dominant category: V2–V3
(Slightly faster at night)
Broader distribution, indicating behavioral variability.

INDEX

FROM MOTION TO SPATIAL MODEL

1- video tracking



2- Create a top-down planar model based on reference dimensions from the video

3- Convert each person's data into planar point data (x, y, distance difference, speed)

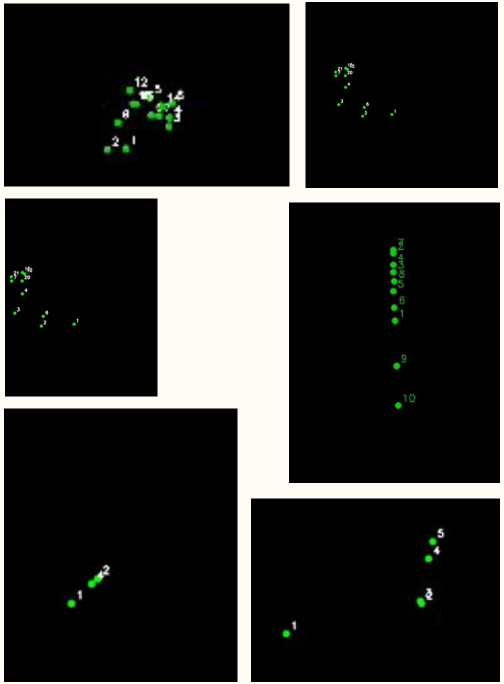
C0300_tracking_data

frame	person_id	cx_px	cy_px	wx_m	wy_m	dist_m	speed_m_s
0	1	447.74725341796900	639.5787506103520	1.2553126811981200	-2.5730667114257800		
0	2	1520.9119262695300	665.4809265136720	1.2160025835037200	-0.5088284015655520		
0	3	989.2443237304690	659.8147888183590	1.2179629802703900	-1.0896236896514900		
1	2	1519.2965698242200	662.459716796875	1.2188557386398300	-0.5177376866340640	0.009354990895918180	0.5607387150400510
1	3	984.5310668945310	658.5839538574220	1.2195701599121100	-1.1006450653076200	0.01113794180961010	0.6676088996769280
1	4	1135.4757690429700	665.0658264160160	1.212830901145940	-0.8761733770370480	0.005710183817566690	0.34226876029370800
1	5	683.5226745605470	659.8402099609380	1.2142866849899300	-1.6943503618240400	0.02439041701908520	1.4619630580870200
1	6	542.7974090576170	651.4317626953130	1.2288511991500900	-2.151780605316160	0.0034268645444084500	0.20540646619830900

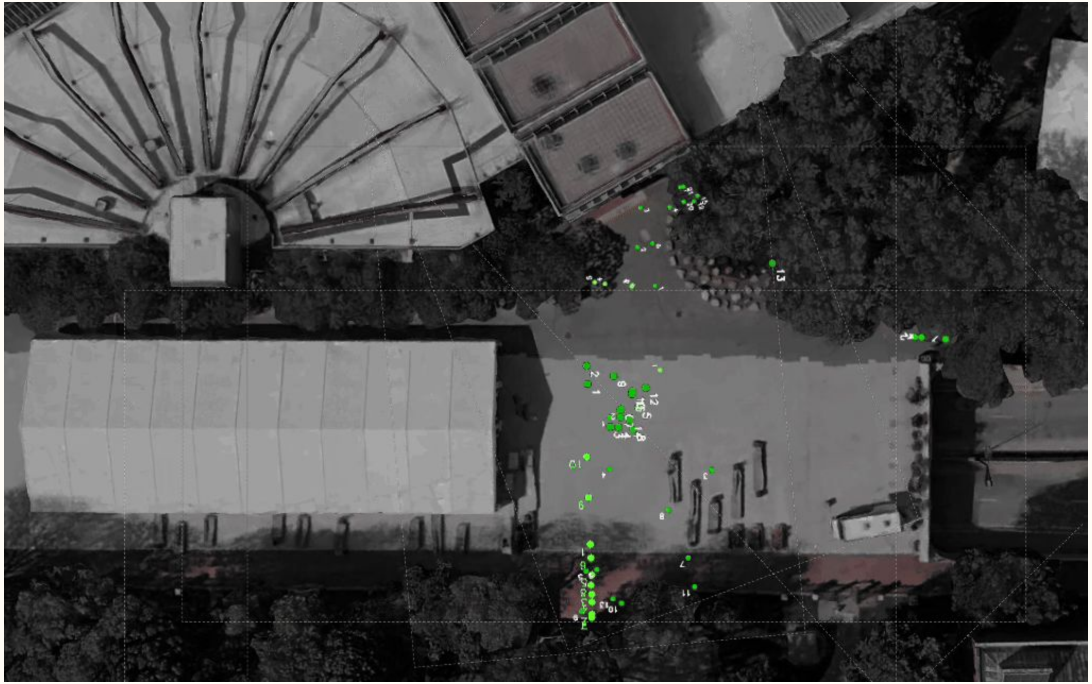
4- Create Scene Top-down Video

FROM MOTION TO SPATIAL MODEL

Top-down Videos



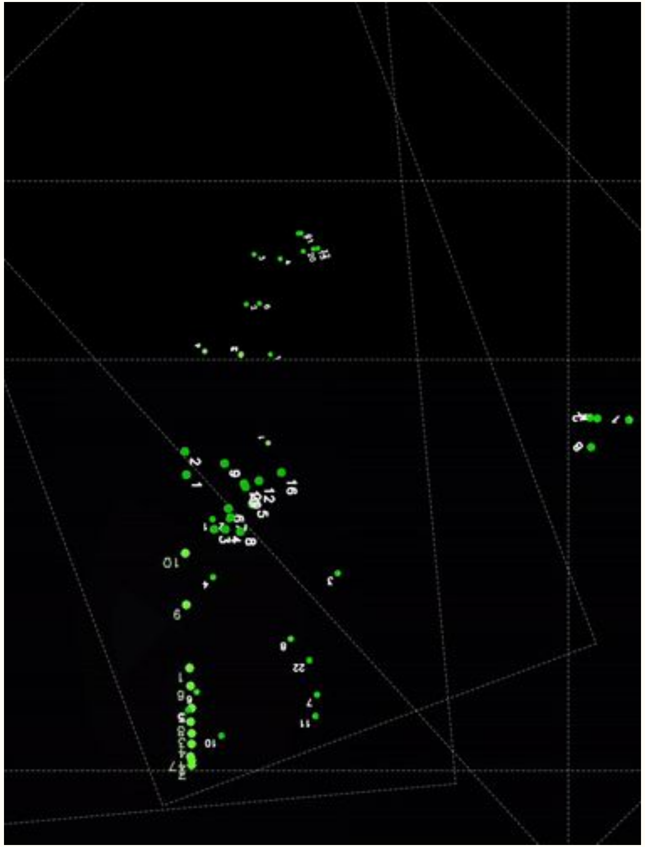
Composite into satellite image



Using actual distances on the map as a reference scale, overlay top-down videos of each scene onto real satellite imagery.

FROM MOTION TO SPATIAL MODEL

Combined Top- Down Video



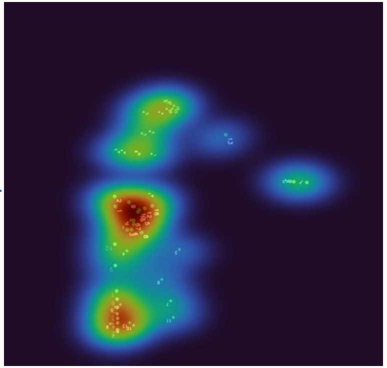
Coding Analytic

```
video_path = "/Users/raine/Desktop/X2.mov"      # input video
output_path = "/Users/raine/Desktop/X2_density_heatmap.mp4"  # output video

# heatmap resolution (internal)
heatmap_w, heatmap_h = 900, 900

# how long traces stay on screen (0–1, closer to 1 = more memory)
decay = 0.95

# how strong each point contributes to the heatmap
presence_strength = 20.0

# how big each point appears (in pixels on heatmap grid)
point_radius = 20

# gaussian blur to smooth the heatmap
blur_sigma = 35

# blend between heatmap and original frame
alpha_heat = 0.6   # heatmap weight
alpha_frame = 0.4  # original video weight


cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise RuntimeError("Could not open video: {}".format(video_path))

fps = cap.get(cv2.CAP_PROP_FPS)
frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_w, frame_h))

# heatmap buffer (float32 for accumulation)
heatmap = np.zeros((heatmap_h, heatmap_w), dtype=np.float32)

# kernel for expanding each point into a blob
kernel_size = (point_radius * 2 + 1, point_radius * 2 + 1)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, kernel_size)
```

PRESENCE

Heat Maps

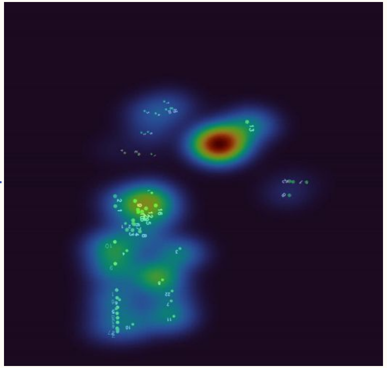
Presence/Density HeatMap



```
video_path = "/Users/raine/Desktop/X2.mov"
output_path = "/Users/raine/Desktop/X2_flow_speed_heatmap.mp4"

# internal heatmap resolution (can be lower than original)
heatmap_w, heatmap_h = 900, 900

# how much history to keep (0–1)
decay = 0.94

# scale factor for how much each frame's motion contributes
motion_strength = 4.0

# ignore very small motions (noise threshold in pixels/frame)
motion_threshold = 0.5

# smoothing of the heat field
blur_sigma = 25

# blending with original video
alpha_heat = 0.6
alpha_frame = 0.4


cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise RuntimeError("Could not open video: {}".format(video_path))

fps = cap.get(cv2.CAP_PROP_FPS)
frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_w, frame_h))

ret, first_frame = cap.read()
if not ret:
    cap.release()
    raise RuntimeError("Could not read first frame from video.")

# resize first frame to heatmap space and convert to gray
prev_small = cv2.resize(first_frame, (heatmap_w, heatmap_h))
prev_gray = cv2.cvtColor(prev_small, cv2.COLOR_BGR2GRAY)
```

SPEED

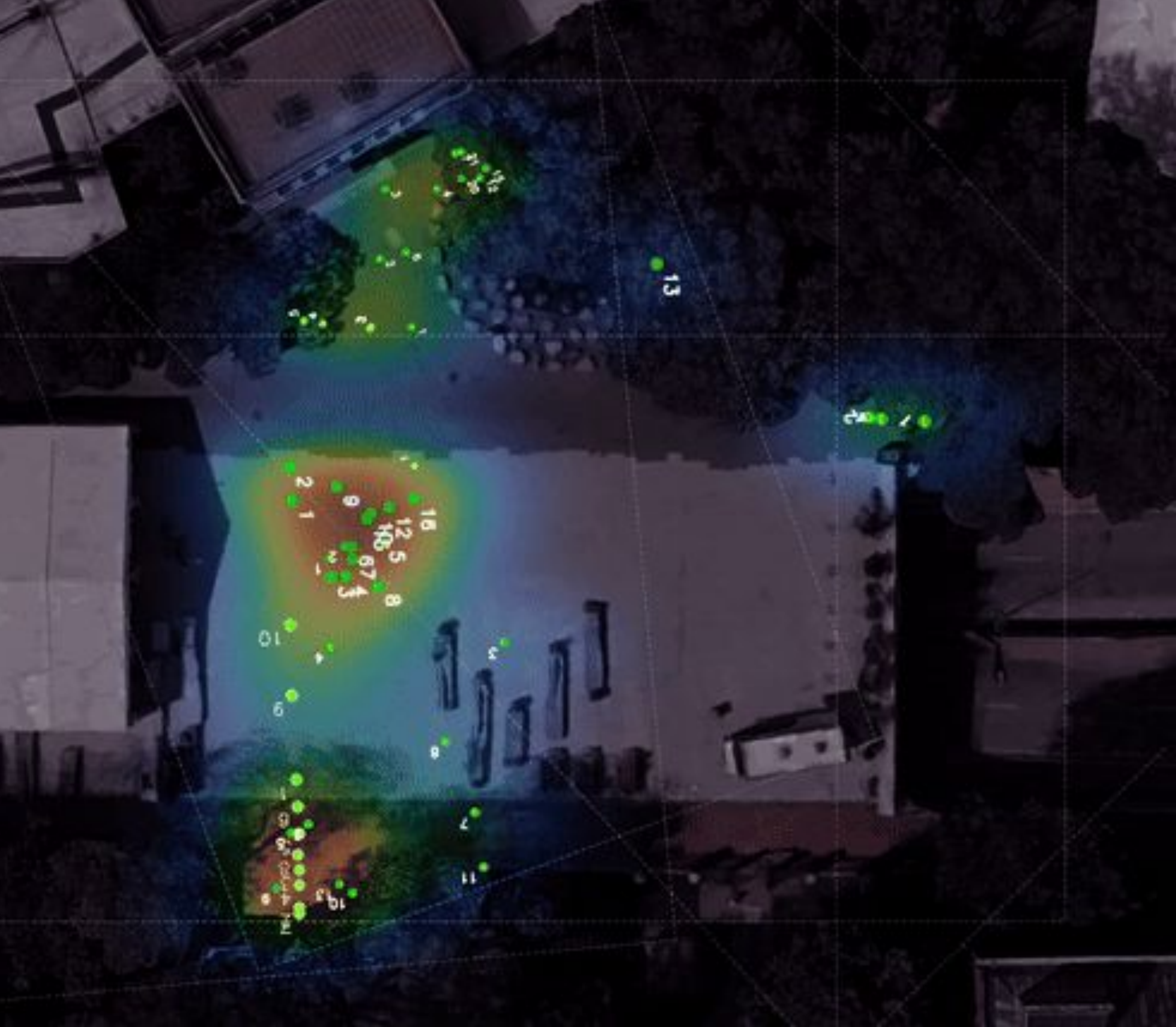
Speed-Weighted HeatMap



HEAT MAPS: PRESENCE AND DENSITY

FROM MOTION TO SPATIAL MODEL

```
# heatmap resolution (internal)
heatmap_w, heatmap_h = 900, 900

# how long traces stay on screen (0-1, closer to 1 = more
memory)
decay = 0.95

# how strong each point contributes to the heatmap
presence_strength = 20.0

# how big each point appears (in pixels on heatmap grid)
point_radius = 20

# gaussian blur to smooth the heatmap
blur_sigma = 35

# blend between heatmap and original frame
alpha_heat = 0.6   # heatmap weight
alpha_frame = 0.4  # original video weight


cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise RuntimeError("Could not open video:
{}".format(video_path))

fps = cap.get(cv2.CAP_PROP_FPS)
frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps,
(frame_w, frame_h))

# heatmap buffer (float32 for accumulation)
heatmap = np.zeros((heatmap_h, heatmap_w),
dtype=np.float32)
```



HEAT MAPS: FLOW SPEED

FROM MOTION TO SPATIAL MODEL

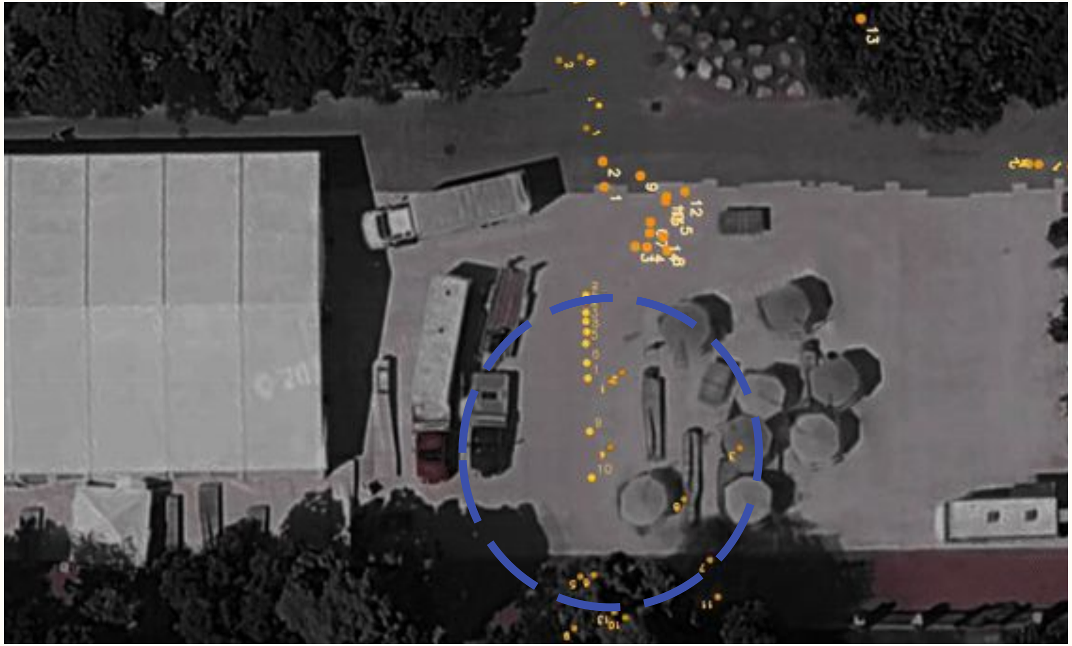
```
# internal heatmap resolution (can be lower than original)
heatmap_w, heatmap_h = 900, 900
# how much history to keep (0-1)
decay = 0.94
# scale factor for how much each frame's motion contributes
motion_strength = 4.0
# ignore very small motions (noise threshold in pixels/frame)
motion_threshold = 0.5
# smoothing of the heat field
blur_sigma = 25
# blending with original video
alpha_heat = 0.6
alpha_frame = 0.4
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
   raise RuntimeError("Could not open video: {}".format(video_path))
fps = cap.get(cv2.CAP_PROP_FPS)
frame_w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_w, frame_h))
ret, first_frame = cap.read()
if not ret:
   cap.release()
   raise RuntimeError("Could not read first frame from video")
# resize first frame to heatmap space and convert to gray
prev_small = cv2.resize(first_frame, (heatmap_w, heatmap_h))
prev_gray = cv2.cvtColor(prev_small, cv2.COLOR_BGR2GRAY)
heatmap = np.zeros((heatmap_h, heatmap_w), dtype=np.float32)
while True:
   ret, frame = cap.read()
   if not ret:
       break

   # work at heatmap resolution
   small = cv2.resize(frame, (heatmap_w, heatmap_h))
   gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
```

Science Center Entrance

V1–V2 lingering overlapping V3–V5 fast flows (e.g. entrance waiting + scooters).

Cross-directional paths

Two or more strong desire lines intersect at oblique or perpendicular angles.



Geometric constriction (Harvard Yard gate)

gates, narrow passages, seating clusters forming bottlenecks

INDEX

stopping x passing conflict

Challenge: Fast commuters and slow lingerers occupy the same space.

Goal: Allow both behaviors to happen safely and comfortably.

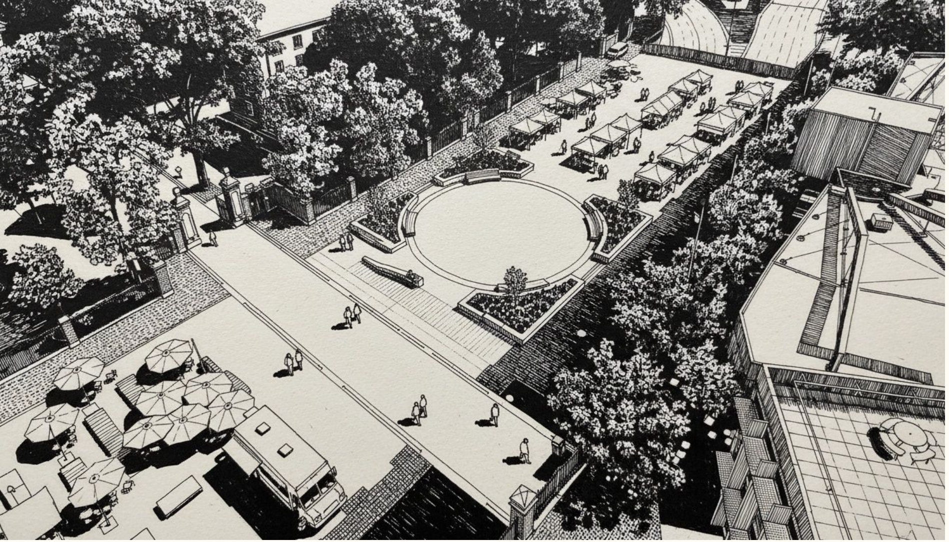
strategies

waiting pockets,
walk-through lanes,
micro-seating elements

cross-directional conflict

Challenge: Several desire lines intersect, creating unpredictable movement patterns.

Goal: Help people intuitively understand where the dominant flows move.



strategies

differentiated pavement tones,
directional ground lighting
(automated feedback loop)

small scale landscape
elements, low seating,
clearer heart of the plaza

geometric constriction

Challenge: A narrow opening forces users
to slow down and compress.

Goal: Reduce crowding, improve predictability,
and make the slowdown feel intentional.

strategies

pre-gate funnel zone - lighting, texture, greenery
secondary micropath
widen entry area